

Conjugate Direction Methods

Conjugate Direction Algorithm,
Conjugate Gradient Algorithm,
Non-quadratic Formulas:

Hestenes-Stiefel, Polak Ribiere, Fletcher Reeves

ADE25232E - Lecture 6

2026

Conjugate Direction Methods

Conjugate direction methods can be viewed as intermediate between **steepest descent** and **Newton's method**.

They have the following important properties:

1. Quadratic problems in n variables can be solved in at most n steps.
2. The conjugate gradient implementation does not require explicit Hessian evaluations.
3. Matrix inversion is not required, and there is no need to store a full $n \times n$ matrix.

Definition. Let Q be a real symmetric $n \times n$ matrix. The directions

$$d^{(0)}, d^{(1)}, \dots, d^{(m)}$$

are called **Q -conjugate** or **Q -orthogonal** if

$$(d^{(i)})^T Q d^{(j)} = 0, \quad \text{for all } i \neq j.$$

Linear Independence of Q -Conjugate Directions

Lemma. Let Q be a symmetric positive definite $n \times n$ matrix. If $d^{(0)}, d^{(1)}, \dots, d^{(k)}$ with $k \leq n - 1$ are nonzero and Q -conjugate, then they are linearly independent.

Proof. Suppose

$$\alpha_0 d^{(0)} + \alpha_1 d^{(1)} + \dots + \alpha_k d^{(k)} = 0.$$

Premultiplying by $(d^{(j)})^T Q$, we get

$$\alpha_0 (d^{(j)})^T Q d^{(0)} + \alpha_1 (d^{(j)})^T Q d^{(1)} + \dots + \alpha_k (d^{(j)})^T Q d^{(k)} = 0.$$

By Q -conjugacy, all terms vanish except the $i = j$ term, so

$$\alpha_j (d^{(j)})^T Q d^{(j)} = 0.$$

Since Q is positive definite and $d^{(j)} \neq 0$,

$$(d^{(j)})^T Q d^{(j)} > 0.$$

Hence $\alpha_j = 0$ for all j , and therefore $d^{(0)}, d^{(1)}, \dots, d^{(k)}$ are linearly independent.

Example 1: Orthogonality vs Q-Orthogonality

Consider the vectors

$$d^{(1)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad d^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

These vectors are orthogonal in the standard sense since

$$(d^{(1)})^T d^{(2)} = 0.$$

Now let

$$Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Then

$$(d^{(1)})^T Q d^{(2)} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1 \neq 0.$$

Conclusion: Orthogonality does not imply Q-conjugacy(Q-Orthogonality)

Example 2: Constructing Q -Conjugate Directions

Let

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \quad d^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

We look for $d^{(1)} = \begin{bmatrix} a \\ b \end{bmatrix}$ such that

$$(d^{(0)})^T Q d^{(1)} = 0.$$

Compute:

$$\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = 2a + b = 0.$$

Choose $a = 1$, then $b = -2$, so

$$d^{(1)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Thus, $d^{(0)}$ and $d^{(1)}$ are Q -conjugate.

Example 3: Constructing Q -Conjugate Directions

Let

$$Q = \begin{bmatrix} 3 & 0 & 1 \\ 0 & 4 & 2 \\ 1 & 2 & 3 \end{bmatrix}.$$

Construct a set of Q -conjugate directions in $\mathbb{R}^3$.

Steps:

- ▶ Choose an arbitrary nonzero vector $d^{(0)}$
- ▶ Find $d^{(1)}$ such that

$$(d^{(0)})^T Q d^{(1)} = 0$$

- ▶ Find $d^{(2)}$ such that

$$(d^{(0)})^T Q d^{(2)} = 0, \quad (d^{(1)})^T Q d^{(2)} = 0$$

Hint: This is analogous to Gram–Schmidt orthogonalization with respect to the inner product

$$\langle x, y \rangle_Q = x^T Q y.$$

Solution (Step 1)

Choose

$$d^{(0)} = (1, 0, 0)^T.$$

To find a Q -conjugate direction $d^{(1)}$, let

$$d^{(1)} = (a, b, c)^T.$$

We impose

$$(d^{(0)})^T Q d^{(1)} = 0.$$

Since

$$(1, 0, 0) \begin{bmatrix} 3 & 0 & 1 \\ 0 & 4 & 2 \\ 1 & 2 & 3 \end{bmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = 3a + c,$$

the condition becomes

$$3a + c = 0.$$

Choose $a = 1$ and $b = 0$. Then $c = -3$, so we obtain

$$d^{(1)} = (1, 0, -3)^T.$$

Solution (Step 2)

Now let $d^{(2)} = (x, y, z)^T$. We require

$$(d^{(0)})^T Q d^{(2)} = 0, \quad (d^{(1)})^T Q d^{(2)} = 0.$$

The first condition gives

$$(1, 0, 0) Q (x, y, z)^T = 3x + z = 0.$$

For the second one, using $d^{(1)} = (1, 0, -3)^T$,

$$(1, 0, -3) Q (x, y, z)^T = 0.$$

Since $Q(x, y, z)^T = (3x + z, 4y + 2z, x + 2y + 3z)^T$, we get

$$3y + 4z = 0.$$

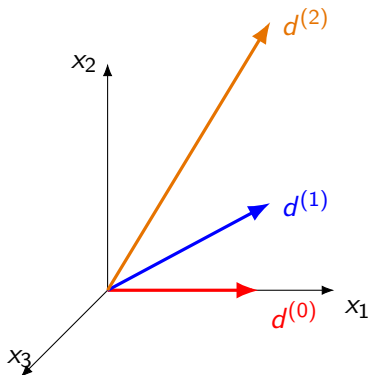
A convenient choice is $z = -3$. Then $x = 1$, $y = 4$, and hence

$$d^{(2)} = (1, 4, -3)^T.$$

Therefore, $d^{(0)} = (1, 0, 0)^T$, $d^{(1)} = (1, 0, -3)^T$, $d^{(2)} = (1, 4, -3)^T$ form a set of Q -conjugate directions.

Remark: This is analogous to Gram-Schmidt with $\langle x, y \rangle_Q = x^T Q y$.

Q-Conjugate Directions in $\mathbb{R}^3$



$$Q = \begin{bmatrix} 3 & 0 & 1 \\ 0 & 4 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$$d^{(0)} = (1, 0, 0)^T, \quad d^{(1)} = (1, 0, -3)^T, \quad d^{(2)} = (1, 4, -3)^T$$

Conjugate Directions Algorithms

In the conjugate directions method, we aim to construct a sequence of **Q -conjugate directions** $d_0, d_1, \dots, d_n$.

Instead of searching along standard coordinate axes, this approach utilizes the geometric structure of the objective function by ensuring that distinct direction vectors satisfy the conjugacy condition:

$$d_i^T Q d_j = 0 \quad \text{or equivalently,} \quad \langle d^{(i)}, d^{(j)} \rangle_Q = 0 \quad (i \neq j).$$

These are linear constraints on d_k , hence we are implicitly solving a **linear system**.

Constructing Q -conjugate directions is equivalent to solving linear systems that enforce orthogonality.

1. Conjugate Direction Algorithm (CDA)

We now present the conjugate direction algorithm for minimizing the quadratic function of n variables $f(x) = \frac{1}{2}x^T Qx - x^T b$, where $Q = Q^T > 0$, $x \in \mathbb{R}^n$.

Goal: Find the minimizer x^* , which satisfies $\nabla f(x) = Qx - b = 0$ or $Qx = b$.

Given $x^{(0)}$ and Q -conjugate directions $d^{(0)}, \dots, d^{(n-1)}$:

$$g^{(k)} = \nabla f(x^{(k)}) = Qx^{(k)} - b,$$

$$\alpha_k = -\frac{g^{(k)T} d^{(k)}}{d^{(k)T} Q d^{(k)}},$$

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$$

Convergence theorem: For any starting point $x^{(0)}$, the conjugate direction method finds the exact solution x^* in at most n steps.

1. Conjugate Direction Method

We consider minimizing the quadratic function of n variables:

$$f(x) = \frac{1}{2}x^T Qx - x^T b,$$

where $Q = Q^T > 0$, $x \in \mathbb{R}^n$.

Goal: Find the minimizer x^* , which satisfies $\nabla f(x) = Qx - b = 0$ or $Qx = b$.

The Key Idea: Given a starting point $x^{(0)}$ and n Q -conjugate directions $d^{(0)}, \dots, d^{(n-1)}$, the algorithm generates a sequence of points:

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$$

Instead of a fixed step size, α_k is determined via an **exact line search**. It is optimally chosen to minimize f along the direction $d^{(k)}$.

Convergence Theorem

For any starting point $x^{(0)}$, the conjugate direction method converges to the exact solution x^* in **at most n steps**.

Conjugate Direction Algorithm (CDA)

Given: A set of Q -conjugate directions $d^{(0)}, d^{(1)}, \dots, d^{(n-1)}$ is already available.

Step 1. Choose $x^{(0)}$ and set $k = 0$.

Step 2. $g^{(k)} = \nabla f(x^{(k)})$. If $g^{(k)} = 0$, stop.

Step 3. $\alpha_k = -\frac{g^{(k)T} d^{(k)}}{d^{(k)T} Q d^{(k)}}.$

Step 4. $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$

Step 5. Set $k = k + 1$. If $k = n$, stop; otherwise go to Step 2.

Example 4: Conjugate Direction Method

Find the minimizer of $f(x_1, x_2) = \frac{1}{2}x^T \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} x - x^T \begin{bmatrix} -1 \\ 1 \end{bmatrix}$, $x \in \mathbb{R}^2$,
using the conjugate direction method with the initial point

$$x^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T,$$

and the Q -conjugate directions

$$d^{(0)} = \begin{bmatrix} 1 & 0 \end{bmatrix}^T, \quad d^{(1)} = \begin{bmatrix} -\frac{3}{8} & \frac{3}{4} \end{bmatrix}^T.$$

We have $g^{(0)} = -b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, and hence

$$\alpha_0 = -\frac{g^{(0)T} d^{(0)}}{d^{(0)T} Q d^{(0)}} = -\frac{\begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}}{\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}} = -\frac{1}{4}.$$

Thus,

$$x^{(1)} = x^{(0)} + \alpha_0 d^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{4} \\ 0 \end{bmatrix}.$$

To find $x^{(2)}$, we compute

$$g^{(1)} = Qx^{(1)} - b = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -\frac{1}{4} \\ 0 \end{bmatrix} - \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -\frac{3}{2} \end{bmatrix}.$$

Then

$$\alpha_1 = -\frac{g^{(1)T}d^{(1)}}{d^{(1)T}Qd^{(1)}} = -\frac{\begin{bmatrix} 0 & -\frac{3}{2} \end{bmatrix} \begin{bmatrix} -\frac{3}{8} \\ \frac{3}{4} \end{bmatrix}}{\begin{bmatrix} -\frac{3}{8} & \frac{3}{4} \end{bmatrix} \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -\frac{3}{8} \\ \frac{3}{4} \end{bmatrix}} = 2.$$

Therefore,

$$x^{(2)} = x^{(1)} + \alpha_1 d^{(1)} = \begin{bmatrix} -\frac{1}{4} \\ 0 \end{bmatrix} + 2 \begin{bmatrix} -\frac{3}{8} \\ \frac{3}{4} \end{bmatrix} = \begin{bmatrix} -1 \\ \frac{3}{2} \end{bmatrix}.$$

Because f is a quadratic function in two variables, we obtain

$$x^{(2)} = x^*.$$

From Conjugate Direction to Conjugate Gradient

In the basic conjugate direction method, we minimize

$$f(x) = \frac{1}{2}x^T Qx - x^T b$$

by moving along Q -conjugate directions $d^{(0)}, d^{(1)}, \dots, d^{(n-1)}$.

If these directions are **available**, the method reaches the exact solution in at most n steps.

The Practical Limitation:

We rarely know these directions in advance. Pre-computing a full set of Q -conjugate vectors is computationally expensive for large systems.

The Goal: Can we build these directions iteratively, using only the information computed during the search?

2. The Conjugate Gradient (CG) Method

Instead of requiring all directions beforehand, the conjugate gradient method constructs them "on the fly": dynamically during each iteration.

We start at an initial point $x^{(0)}$ and choose the first direction simply as the **steepest descent direction**:

$$d^{(0)} = -g^{(0)}$$

where $g^{(0)} = \nabla f(x^{(0)}) = Qx^{(0)} - b$.

Then, we perform an exact line search to find the next point:

$$x^{(1)} = x^{(0)} + \alpha_0 d^{(0)}$$

But what about the next direction ($d^{(1)}$)? If we just take $-g^{(1)}$, we lose the Q -conjugate property.

Generating Conjugate Directions Iteratively

To maintain the n -step convergence guarantee, the new direction must be Q -conjugate to the previous one, while still descending the gradient.

We construct the new direction by combining the current steepest descent with a correction term from the previous direction:

$$d^{(k+1)} = -g^{(k+1)} + \beta_k d^{(k)}$$

Here:

- ▶ $-g^{(k+1)}$: Ensures we move downhill (Gradient part).
- ▶ $\beta_k d^{(k)}$: Corrects the path to be Q -conjugate (Conjugate part).

This elegant combination of simplicity and exact finite-step convergence gives the **Conjugate Gradient** method its name.

The Choice of β_k

In the previous slide, we defined the update rule:

$$d^{(k+1)} = -g^{(k+1)} + \beta_k d^{(k)}$$

To ensure that our new direction $d^{(k+1)}$ is strictly Q -conjugate to the previous direction $d^{(k)}$, we must satisfy:

$$d^{(k+1)T} Q d^{(k)} = 0$$

By substituting our update rule into this condition and solving for β_k , we get the exact coefficient required:

$$\beta_k = \frac{g^{(k+1)T} Q d^{(k)}}{d^{(k)T} Q d^{(k)}}$$

Observe that each new direction is constructed from:

- ▶ the current gradient, and
- ▶ the previous search direction.

Now, no precomputed set of conjugate directions is required.

Summary

Basic Conjugate Direction Method:

- ▶ assumes a set of Q -conjugate directions is already available;
- ▶ converges in at most n steps.

Conjugate Gradient Method:

- ▶ constructs these directions iteratively;
- ▶ starts with steepest descent;
- ▶ modifies each new gradient direction to preserve conjugacy.

Hence, the conjugate gradient method is a practical version of the conjugate direction method.

Conjugate Gradient Algorithm (CGA)

Step 1. Choose $x^{(0)}$, set $k = 0$.

Step 2. $g^{(0)} = \nabla f(x^{(0)})$. If $g^{(0)} = 0$, stop; else $d^{(0)} = -g^{(0)}$.

Step 3. $\alpha_k = -\frac{g^{(k)T} d^{(k)}}{d^{(k)T} Q d^{(k)}}.$

Step 4. $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$

Step 5. $g^{(k+1)} = \nabla f(x^{(k+1)})$. If $g^{(k+1)} = 0$, stop.

Step 6. $\beta_k = \frac{g^{(k+1)T} Q d^{(k)}}{d^{(k)T} Q d^{(k)}}.$

Step 7. $d^{(k+1)} = -g^{(k+1)} + \beta_k d^{(k)}.$

Step 8. $k = k + 1$; go to Step 3.

Algorithmic Efficiency: Step Length (α_k)

Simplifying the α_k Formula

The general formula given in the algorithm is:

$$\alpha_k = -\frac{g^{(k)T} d^{(k)}}{d^{(k)T} Q d^{(k)}}$$

The new direction vector is updated as $d^{(k)} = -g^{(k)} + \beta_{k-1} d^{(k-1)}$. Due to the **exact line search**, the current gradient is orthogonal to the previous direction ($g^{(k)T} d^{(k-1)} = 0$).

Substituting this into the numerator yields:

$$-g^{(k)T} d^{(k)} = -g^{(k)T} (-g^{(k)} + \beta_{k-1} d^{(k-1)}) = g^{(k)T} g^{(k)}$$

Conclusion: In practical optimization software, the numerator is simply computed as the inner product of the gradient with itself:

$$\alpha_k = \frac{g^{(k)T} g^{(k)}}{d^{(k)T} Q d^{(k)}}$$

Algorithmic Efficiency: Gradient Update Shortcut

Gradient Update ($g^{(k+1)}$) and Computational Complexity

By definition, the gradient at the new point is:

$$g^{(k+1)} = \nabla f(x^{(k+1)}) = Qx^{(k+1)} - b$$

Substituting the position update rule $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$ yields:

$$g^{(k+1)} = Q(x^{(k)} + \alpha_k d^{(k)}) - b = \underbrace{(Qx^{(k)} - b)}_{g^{(k)}} + \alpha_k Qd^{(k)} = g^{(k)} + \alpha_k Qd^{(k)}$$

Why use this alternate form?

- ▶ Computing $Qx^{(k+1)} - b$ directly requires a costly new matrix-vector multiplication ($\mathcal{O}(n^2)$).
- ▶ However, the vector $Qd^{(k)}$ is **already computed** when evaluating α_k in the previous step.
- ▶ Reusing this stored term reduces the gradient update to a simple vector addition, dropping the complexity to just $\mathcal{O}(n)$!

Example 5:

Consider

$$f(x) = \frac{5}{2}x_1^2 + \frac{1}{2}x_2^2 + 2x_1x_2 - 3x_1 - x_2,$$

where $x = (x_1, x_2)^T \in \mathbb{R}^2$.

- Express $f(x)$ in the form $f(x) = \frac{1}{2}x^T Qx - x^T b$.
- Compute the minimizer analytically from Q and b .
- Find the minimizer of f using the conjugate gradient algorithm starting from $x^{(0)} = (0, 0)^T$, and compare it with the analytical answer in part (b).

a. Quadratic Form

$$\nabla f(x) = \begin{bmatrix} 5x_1 + 2x_2 - 3 \\ 2x_1 + x_2 - 1 \end{bmatrix}, \quad Q = \begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$f(x) = \frac{1}{2} \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

b. Analytical Solution

The global minimum of a strictly convex quadratic function occurs where

$$\nabla f(x) = 0,$$

which implies

$$Qx = b.$$

So we solve the linear system

$$\begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

From the second row,

$$2x_1 + x_2 = 1 \implies x_2 = 1 - 2x_1.$$

Substituting into the first row,

$$5x_1 + 2(1 - 2x_1) = 3 \implies x_1 = 1.$$

Hence, $x_2 = 1 - 2(1) = -1$.

Therefore, the analytical minimizer is $x^* = [1, -1]^T$.

c. Iteration 0

Starting point: $x^{(0)} = [0, 0]^T$.

1. Initial Gradient and Direction:

$$g^{(0)} = Qx^{(0)} - b = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ -1 \end{bmatrix}$$

$$d^{(0)} = -g^{(0)} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

2. Step Length (α_0):

$$\alpha_0 = \frac{g^{(0)T} g^{(0)}}{d^{(0)T} Q d^{(0)}} = \frac{10}{\begin{bmatrix} 3 & 1 \end{bmatrix} \begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix}} = \frac{10}{58} = \frac{5}{29}$$

3. Update Position ($x^{(1)}$):

$$x^{(1)} = x^{(0)} + \alpha_0 d^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \frac{5}{29} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 15/29 \\ 5/29 \end{bmatrix}$$

Iteration 1

1. New Gradient ($g^{(1)}$):

$$g^{(1)} = g^{(0)} + \alpha_0 Q d^{(0)} = \begin{bmatrix} -3 \\ -1 \end{bmatrix} + \frac{5}{29} \begin{bmatrix} 17 \\ 7 \end{bmatrix} = \begin{bmatrix} -2/29 \\ 6/29 \end{bmatrix}$$

2. Calculate β_0 :

$$\beta_0 = \frac{g^{(1)T} Q d^{(0)}}{d^{(0)T} Q d^{(0)}} = \frac{\begin{bmatrix} -2/29 & 6/29 \end{bmatrix} \begin{bmatrix} 17 \\ 7 \end{bmatrix}}{58} = \frac{8/29}{58} = \frac{4}{841}$$

3. New Direction ($d^{(1)}$):

$$d^{(1)} = -g^{(1)} + \beta_0 d^{(0)} = \begin{bmatrix} 2/29 \\ -6/29 \end{bmatrix} + \frac{4}{841} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \frac{10}{841} \begin{bmatrix} 7 \\ -17 \end{bmatrix}$$

Iteration 1

Are the directions dynamically Q -conjugate?

Let's check if our new direction $d^{(1)}$ is Q -orthogonal to $d^{(0)}$:

$$\begin{aligned} d^{(0)T} Q d^{(1)} &= \begin{bmatrix} 3 & 1 \end{bmatrix} \begin{bmatrix} 5 & 2 \\ 2 & 1 \end{bmatrix} \left(\frac{10}{841} \begin{bmatrix} 7 \\ -17 \end{bmatrix} \right) \\ &= \frac{10}{841} \begin{bmatrix} 17 & 7 \end{bmatrix} \begin{bmatrix} 7 \\ -17 \end{bmatrix} = \frac{10}{841} (119 - 119) = \mathbf{0} \end{aligned}$$

Dynamic conjugacy is successfully achieved!

4. Step Length (α_1) and Final Position ($x^{(2)}$):

$$\alpha_1 = \frac{g^{(1)T} g^{(1)}}{d^{(1)T} Q d^{(1)}} = \frac{40/841}{5800/841^2} = \frac{29}{5}$$

$$x^{(2)} = x^{(1)} + \alpha_1 d^{(1)} = \begin{bmatrix} 15/29 \\ 5/29 \end{bmatrix} + \frac{29}{5} \left(\frac{10}{841} \begin{bmatrix} 7 \\ -17 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

3. CGA for Non-Quadratic Problems

- ▶ For a quadratic function, the matrix Q , that is, the Hessian of the quadratic, is constant.
- ▶ However, for a general nonlinear function, the Hessian is not constant and must be re-evaluated at each iteration of the algorithm.
- ▶ This can be computationally very expensive.
- ▶ Therefore, it is desirable to develop an efficient version of the conjugate gradient algorithm that avoids evaluating the Hessian at every step.

Motivation: We aim to construct conjugate directions using only gradient information, without explicitly using the Hessian. This leads to different formulas for the coefficient β_k , which define practical nonlinear conjugate gradient methods.

The Hestenes–Stiefel Formula

Recall that for the quadratic case

$$\beta_k = \frac{g^{(k+1)T} Q d^{(k)}}{d^{(k)T} Q d^{(k)}}.$$

Also,

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$$

Multiplying both sides by Q , and using $g^{(k)} = Qx^{(k)} - b$, we obtain

$$g^{(k+1)} = g^{(k)} + \alpha_k Q d^{(k)}.$$

Hence,

$$Q d^{(k)} = \frac{g^{(k+1)} - g^{(k)}}{\alpha_k}.$$

Substituting this into the formula for β_k , we get

$$\beta_k^{HS} = \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{d^{(k)T} (g^{(k+1)} - g^{(k)})}.$$

This is called the **Hestenes–Stiefel formula**.

The Logic Behind Hestenes-Stiefel (HS): $\beta_k^{HS} = \frac{g^{(k+1)T}(g^{(k+1)} - g^{(k)})}{d^{(k)T}(g^{(k+1)} - g^{(k)})}$.

Robustness via Inherent Conjugacy

The core advantage of the HS formula lies in its robustness when exact line searches are practically impossible (e.g., in nonlinear problems).

The Magic Property

Let $y_k = g^{(k+1)} - g^{(k)}$. By algebraic design, the HS formula forces the new direction to satisfy:

$$d^{(k+1)T} y_k = 0 \quad \text{for **any** step length } \alpha_k.$$

This means the HS formula acts as an "auto-correct" mechanism. It guarantees that local Q -conjugacy is maintained **independently** of how perfect the line search is!

From Hestenes-Stiefel to Polak-Ribière

Assume we use an **exact line search**. This implies that the new gradient is orthogonal to the search direction:

$$g^{(k+1)T} d^{(k)} = 0 \quad \text{and} \quad g^{(k)T} d^{(k-1)} = 0.$$

Let's examine the denominator of the HS formula:

$$d^{(k)T} (g^{(k+1)} - g^{(k)}) = \underbrace{d^{(k)T} g^{(k+1)}}_0 - d^{(k)T} g^{(k)} = -d^{(k)T} g^{(k)}.$$

Substitute the search direction $d^{(k)} = -g^{(k)} + \beta_{k-1} d^{(k-1)}$:

$$\begin{aligned} -d^{(k)T} g^{(k)} &= -(-g^{(k)} + \beta_{k-1} d^{(k-1)})^T g^{(k)} \\ &= g^{(k)T} g^{(k)} - \beta_{k-1} \underbrace{d^{(k-1)T} g^{(k)}}_0. \end{aligned}$$

Thus, the denominator simplifies exactly to $g^{(k)T} g^{(k)}$, yielding the following PR formula:

$$\beta_k^{PR} = \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{g^{(k)T} g^{(k)}}.$$

The Logic Behind Polak-Ribière (PR): $\beta_k^{PR} = \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{g^{(k)T} g^{(k)}}$.

The Shared Magic: "Automatic Restart"

Notice that the PR formula **inherits** its exact numerator from the HS formula. This specific design gives both methods a highly favored built-in restart mechanism:

- ▶ If the algorithm takes a bad step and makes little progress, consecutive gradients will be very similar: $g^{(k+1)} \approx g^{(k)}$.
- ▶ As a result, the inherited numerator approaches zero, forcing $\beta_k \approx 0$.
- ▶ When β_k drops to zero, the algorithm "forgets" past directions and resets to the **Steepest Descent** direction.

$$d^{(k+1)} = -g^{(k+1)} + \overset{0}{\cancel{\beta_k}} d^{(k)} = -g^{(k+1)}$$

- ▶ This clears out poor search history and turns the method into **Steepest Descent**, allowing it to recover when it gets **stuck**.

From Polak-Ribière to Fletcher-Reeves

Let's examine the numerator of the **Polak-Ribière** formula:

$$g^{(k+1)T} (g^{(k+1)} - g^{(k)}) = g^{(k+1)T} g^{(k+1)} - g^{(k+1)T} g^{(k)}.$$

If we continue to assume an **exact line search** applied to a quadratic function, consecutive gradients are orthogonal. This gives us another powerful property:

$$g^{(k+1)T} g^{(k)} = 0.$$

Applying this orthogonality condition to the numerator:

$$g^{(k+1)T} g^{(k+1)} - \underbrace{g^{(k+1)T} g^{(k)}}_0 = g^{(k+1)T} g^{(k+1)}.$$

Substituting this simplified numerator back into the PR formula yields the **Fletcher-Reeves formula**:

$$\beta_k^{FR} = \frac{g^{(k+1)T} g^{(k+1)}}{g^{(k)T} g^{(k)}}.$$

The Logic Behind Fletcher-Reeves (FR): $\beta_k^{FR} = \frac{g^{(k+1)T} g^{(k+1)}}{g^{(k)T} g^{(k)}}$.

Simplicity vs. Global Convergence

Unlike HS and PR, the FR formula uses a simpler numerator $(g^{(k+1)T} g^{(k+1)})$ and does **not** inherit the automatic restart property.

The Risk: If it takes bad steps, it can suffer from "**jamming**" (taking infinitely many tiny steps without making any progress).

Why do we still use FR?

- ▶ **Theoretical Guarantee:** FR has mathematically proven **Global Convergence**. Under standard line search conditions, FR is strictly guaranteed to converge to a minimum.
- ▶ **Trade-off:** PR and HS perform much better in practice but lack this strict theoretical global convergence guarantee without external algorithmic modifications.
- ▶ **Computational Efficiency:** It requires the least amount of memory and computation.

Summary: Comparing the Choices of β_k

Let's look at the three formulas side-by-side. Notice how the mathematical structure **evolves** and simplifies based on the algorithmic goals we discussed:

Hestenes–Stiefel (HS): *(Focus on Robustness)*

$$\beta_k^{HS} = \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{d^{(k)T} (g^{(k+1)} - g^{(k)})}$$

Polak–Ribière (PR): *(Focus on Automatic Restart)*

$$\beta_k^{PR} = \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{g^{(k)T} g^{(k)}}$$

Fletcher–Reeves (FR): *(Focus on Global Convergence)*

$$\beta_k^{FR} = \frac{g^{(k+1)T} g^{(k+1)}}{g^{(k)T} g^{(k)}}$$

Conjugate Gradient Algorithm for Non-Quadratic Problems

Step 1. Choose $x^{(0)}$, set $k = 0$.

Step 2. $g^{(0)} = \nabla f(x^{(0)})$. If $g^{(0)} = 0$, stop; else $d^{(0)} = -g^{(0)}$.

Step 3. Choose α_k so as to minimize $f(x^{(k)} + \alpha d^{(k)})$ with respect to α .

Step 4. $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$.

Step 5. $g^{(k+1)} = \nabla f(x^{(k+1)})$. If $g^{(k+1)} = 0$, stop.

Step 6. Compute β_k using one of the formulas: β_k^{HS} , β_k^{PR} , or β_k^{FR} .

Step 7. $d^{(k+1)} = -g^{(k+1)} + \beta_k d^{(k)}$.

Step 8. Set $k = k + 1$; go to Step 3.

Example 6: A Non-Quadratic Function

Consider the function

$$f(x_1, x_2) = x_1^4 + x_2^2 - x_1x_2 - 2x_1.$$

and the initial point

$$x^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Apply the nonlinear conjugate gradient algorithm with exact line search 3 iterations, and compare the three choices

$$\beta_k^{HS}, \quad \beta_k^{PR}, \quad \beta_k^{FR}.$$

Its gradient is

$$\nabla f(x_1, x_2) = \begin{pmatrix} 4x_1^3 - x_2 - 2 \\ 2x_2 - x_1 \end{pmatrix}.$$

Since the function is non-quadratic, the Hessian is not constant:

$$\nabla^2 f(x_1, x_2) = \begin{pmatrix} 12x_1^2 & -1 \\ -1 & 2 \end{pmatrix}.$$

HS Method — Iteration 0

At $x^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, we have $g^{(0)} = \nabla f(x^{(0)}) = \begin{pmatrix} -2 \\ 0 \end{pmatrix}$, $d^{(0)} = -g^{(0)} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$.

Line search along $d^{(0)}$:

$$\phi_0(\alpha) = f(x^{(0)} + \alpha d^{(0)}) = f(2\alpha, 0) = 16\alpha^4 - 4\alpha.$$

Minimizing $\phi_0(\alpha)$ gives

$$\alpha_0 = \left(\frac{1}{16}\right)^{1/3} \approx 0.3969.$$

So,

$$x^{(1)} = x^{(0)} + \alpha_0 d^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \left(\frac{1}{16}\right)^{1/3} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \left(\frac{1}{16}\right)^{1/3} \\ 0 \end{pmatrix} \approx \begin{pmatrix} 0.7937 \\ 0 \end{pmatrix}.$$

Also,

$$f(x^{(1)}) = \left(2 \left(\frac{1}{16}\right)^{1/3}\right)^4 - 2 \left(2 \left(\frac{1}{16}\right)^{1/3}\right) \approx -1.1906.$$

HS Method — Computing β_0^{HS}

Now compute

$$g^{(1)} = \nabla f(x^{(1)}) = \begin{pmatrix} 0 \\ -0.7937 \end{pmatrix}.$$

Then

$$g^{(1)} - g^{(0)} = \begin{pmatrix} 0 \\ -0.7937 \end{pmatrix} - \begin{pmatrix} -2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ -0.7937 \end{pmatrix}.$$

Using the HS formula,

$$\beta_0^{HS} = \frac{g^{(1)T}(g^{(1)} - g^{(0)})}{d^{(0)T}(g^{(1)} - g^{(0)})}.$$

Numerator:

$$g^{(1)T}(g^{(1)} - g^{(0)}) = (0, -0.7937) \cdot (2, -0.7937) = 0.6300.$$

Denominator:

$$d^{(0)T}(g^{(1)} - g^{(0)}) = (2, 0) \cdot (2, -0.7937) = 4.0000.$$

Hence

$$\beta_0^{HS} = \frac{0.6300}{4.0000} = 0.1575.$$

HS Method — Iteration 1

The new search direction is

$$d^{(1)} = -g^{(1)} + \beta_0^{HS} d^{(0)}.$$

So,

$$d^{(1)} = - \begin{pmatrix} 0 \\ -0.7937 \end{pmatrix} + 0.1575 \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.3150 \\ 0.7937 \end{pmatrix}.$$

Now perform line search along $d^{(1)}$:

$$x^{(1)} + \alpha d^{(1)} = \begin{pmatrix} 0.7937 \\ 0 \end{pmatrix} + \alpha \begin{pmatrix} 0.3150 \\ 0.7937 \end{pmatrix}.$$

The minimizing step size is found numerically as

$$\alpha_1 = 0.3863.$$

Thus

$$x^{(2)} = x^{(1)} + \alpha_1 d^{(1)} = \begin{pmatrix} 0.9154 \\ 0.3066 \end{pmatrix}, \quad f(x^{(2)}) = -1.3153.$$

HS Method — Computing β_1^{HS}

At $x^{(2)} = (0.9154, 0.3066)^T$,

$$g^{(2)} = \nabla f(x^{(2)}) = \begin{pmatrix} 0.7620 \\ -0.3022 \end{pmatrix}.$$

Then

$$g^{(2)} - g^{(1)} = \begin{pmatrix} 0.7620 \\ -0.3022 \end{pmatrix} - \begin{pmatrix} 0 \\ -0.7937 \end{pmatrix} = \begin{pmatrix} 0.7620 \\ 0.4915 \end{pmatrix}.$$

Now

$$\beta_1^{HS} = \frac{g^{(2)T}(g^{(2)} - g^{(1)})}{d^{(1)T}(g^{(2)} - g^{(1)})}.$$

Numerator:

$$(0.7620, -0.3022) \cdot (0.7620, 0.4915) = 0.4321.$$

Denominator:

$$(0.3150, 0.7937) \cdot (0.7620, 0.4915) = 0.6301.$$

Hence

$$\beta_1^{HS} = \frac{0.4321}{0.6301} = 0.6858.$$

HS Method — Iteration 2

The next direction is

$$d^{(2)} = -g^{(2)} + \beta_1^{HS} d^{(1)}.$$

So,

$$d^{(2)} = - \begin{pmatrix} 0.7620 \\ -0.3022 \end{pmatrix} + 0.6858 \begin{pmatrix} 0.3150 \\ 0.7937 \end{pmatrix} = \begin{pmatrix} -0.5460 \\ 0.8465 \end{pmatrix}.$$

Line search along $d^{(2)}$ gives

$$\alpha_2 = 0.1311.$$

Therefore,

$$x^{(3)} = x^{(2)} + \alpha_2 d^{(2)} = \begin{pmatrix} 0.9154 \\ 0.3066 \end{pmatrix} + 0.1311 \begin{pmatrix} -0.5460 \\ 0.8465 \end{pmatrix} = \begin{pmatrix} 0.8438 \\ 0.4176 \end{pmatrix}.$$

Also,

$$f(x^{(3)}) = -1.3588.$$

HS Method — Computing β_2^{HS}

At $x^{(3)} = (0.8438, 0.4176)^T$,

$$g^{(3)} = \nabla f(x^{(3)}) = \begin{pmatrix} -0.0144 \\ -0.0086 \end{pmatrix}.$$

Then

$$g^{(3)} - g^{(2)} = \begin{pmatrix} -0.0144 \\ -0.0086 \end{pmatrix} - \begin{pmatrix} 0.7620 \\ -0.3022 \end{pmatrix} = \begin{pmatrix} -0.7764 \\ 0.2936 \end{pmatrix}.$$

Now

$$\beta_2^{HS} = \frac{g^{(3)T}(g^{(3)} - g^{(2)})}{d^{(2)T}(g^{(3)} - g^{(2)})}.$$

Numerator:

$$(-0.0144, -0.0086) \cdot (-0.7764, 0.2936) = 0.0087.$$

Denominator:

$$(-0.5460, 0.8465) \cdot (-0.7764, 0.2936) = 0.6724.$$

Hence

$$\beta_2^{HS} = \frac{0.0087}{0.6724} = 0.0129.$$

Non-quadratic CG: Hestenes–Stiefel (HS) Execution

For $x^{(0)} = (0, 0)^T$, we obtain the following iteration history.

k	$x^{(k)T}$	$f(x^{(k)})$	$d^{(k)T}$	α_k	β_k^{HS}
0	(0.0000, 0.0000)	0.0000	(2.0000, 0.0000)	0.3969	–
1	(0.7937, 0.0000)	–1.1906	(0.3150, 0.7937)	0.3863	0.1575
2	(0.9154, 0.3066)	–1.3153	(–0.5460, 0.8465)	0.1311	0.6858
3	(0.8438, 0.4176)	–1.3588	(0.0074, 0.0195)	–	0.0129

Table: Three iterations of the nonlinear CG method using Hestenes–Stiefel.

Step 1 Analysis:

$$g^{(1)} = (0, -0.7937)^T.$$

$$\beta_0^{HS} = \frac{g^{(1)T}(g^{(1)} - g^{(0)})}{d^{(0)T}(g^{(1)} - g^{(0)})} = \frac{-0.7937(-0.7937 - 0)}{2(0 - (-2))} = \frac{0.63}{4} = 0.1575.$$

Analysis: Does Exact Q-Conjugacy Hold?

Let's test the traditional conjugacy condition $d^{(k+1)T} Q d^{(k)} = 0$ using our non-quadratic example. The Hessian is $Q(x) = \begin{pmatrix} 12x_1^2 & -1 \\ -1 & 2 \end{pmatrix}$.

1. Evaluating Local Q-Conjugacy (Fails):

At $x^{(0)}$, $Q_0 = \begin{pmatrix} 0 & -1 \\ -1 & 2 \end{pmatrix}$.

$$d^{(1)T} Q_0 d^{(0)} = (0.3150 \quad 0.7937) \begin{pmatrix} 0 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = -1.5874 \neq 0$$

Why? Exact local conjugacy is lost because the Hessian matrix is not constant; it constantly changes along the search path.

2. The HS Magic — Inherent Conjugacy (Succeeds):

Instead of a local Q , the Hestenes-Stiefel formula enforces exact conjugacy with respect to the gradient difference $y_k = g^{(k+1)} - g^{(k)}$, which acts as the average curvature over the step.

$$d^{(1)T} y_0 = (0.3150 \quad 0.7937) \begin{pmatrix} 2 \\ -0.7937 \end{pmatrix} = 0.6300 - 0.6300 = 0$$

$$d^{(2)T} y_1 = (-0.5460 \quad 0.8465) \begin{pmatrix} 0.7620 \\ 0.4915 \end{pmatrix} = -0.4160 + 0.4160 = 0$$

Non-quadratic CG: Polak–Ribière (PR) Execution

Using

$$\beta_k^{PR} = \frac{g^{(k+1)T}(g^{(k+1)} - g^{(k)})}{g^{(k)T}g^{(k)}},$$

the same steps can be repeated.

For this example, the PR method gives the following iteration summary:

k	$x^{(k)T}$	$f(x^{(k)})$	$d^{(k)T}$	α_k	β_k^{PR}
0	(0.0000, 0.0000)	0.0000	(2.0000, 0.0000)	0.3969	–
1	(0.7937, 0.0000)	–1.1906	(0.3150, 0.7937)	0.3863	0.1575
2	(0.9154, 0.3066)	–1.3153	(–0.5460, 0.8465)	0.1311	0.6859
3	(0.8438, 0.4176)	–1.3588	(0.0074, 0.0195)	–	0.0129

Table: Iteration summary using Polak–Ribière.

Remark: In this example, PR produces the same numerical iterates as HS for the first three iterations (up to rounding precision due to the exact line search).

Step 1 Analysis:

$$g^{(1)} = (0, -0.7937)^T.$$

$$\beta_0^{PR} = \frac{g^{(1)T}(g^{(1)} - g^{(0)})}{g^{(0)T}g^{(0)}} = \frac{-0.7937(-0.7937 - 0)}{(-2)^2} = \frac{0.63}{4} = 0.1575.$$

Non-quadratic CG: Fletcher–Reeves (FR) Execution

Using

$$\beta_k^{FR} = \frac{g^{(k+1)T} g^{(k+1)}}{g^{(k)T} g^{(k)}},$$

the same steps can be repeated.

For this example, the FR method gives the following iteration summary:

k	$x^{(k)T}$	$f(x^{(k)})$	$d^{(k)T}$	α_k	β_k^{FR}
0	(0.0000, 0.0000)	0.0000	(2.0000, 0.0000)	0.3969	–
1	(0.7937, 0.0000)	–1.1906	(0.3150, 0.7937)	0.3863	0.1575
2	(0.9154, 0.3066)	–1.3153	(–0.4260, 1.1487)	0.1259	1.0665
3	(0.8618, 0.4512)	–1.3567	(–0.1173, –0.0177)	–	0.0199

Table: Iteration summary using Fletcher–Reeves.

Step 1 Analysis:

$$g^{(1)} = (0, -0.7937)^T.$$

$$\beta_0^{FR} = \frac{g^{(1)T} g^{(1)}}{g^{(0)T} g^{(0)}} = \frac{(-0.7937)^2}{(-2)^2} = \frac{0.63}{4} = 0.1575.$$

Comparison of CG Methods: Numerical Summary

After 3 iterations, here is where each variant stands in minimizing $f(x)$:

Method	Final Point $x^{(3)}$	Function Value $f(x^{(3)})$
Hestenes–Stiefel (HS)	$(0.8438, 0.4176)^T$	-1.3588
Polak–Ribière (PR)	$(0.8438, 0.4176)^T$	-1.3588
Fletcher–Reeves (FR)	$(0.8618, 0.4512)^T$	-1.3567

Table: Comparison of the final states at $k = 3$.

Key Observation: HS and PR yield nearly identical numerical trajectories, while FR takes a slightly different path resulting in a higher (worse) function value at this stage.

Why are HS and PR Identical Here?

You might notice that the iterates for HS and PR match exactly. This is not a numerical coincidence, but a mathematical property!

Under **exact line search**, the search direction is orthogonal to the next gradient: $g^{(k+1)T} d^{(k)} = 0$.

Let's examine the denominator of β_k^{HS} :

$$d^{(k)T} (g^{(k+1)} - g^{(k)}) = d^{(k)T} g^{(k+1)} - d^{(k)T} g^{(k)} = 0 - d^{(k)T} g^{(k)}.$$

Since $d^{(k)} = -g^{(k)} + \beta_{k-1} d^{(k-1)}$, substituting this gives:

$$-(-g^{(k)} + \beta_{k-1} d^{(k-1)})^T g^{(k)} = g^{(k)T} g^{(k)} - 0 = \|g^{(k)}\|^2.$$

Conclusion

When exact line search is used, the denominator of HS reduces to $\|g^{(k)}\|^2$. Thus, $\beta_k^{HS} \equiv \beta_k^{PR}$.

Why are the Results for HS and PR Identical?

You might notice that the numerical iterates for HS and PR match exactly in our example. This is **not** a numerical coincidence!

Recall our earlier derivation:

When an **exact line search** is used, the search direction is perfectly orthogonal to the next gradient ($g^{(k+1)T} d^{(k)} = 0$).

As we proved analytically earlier, under this specific orthogonality condition, the complex denominator of the HS formula simplifies exactly to the PR denominator:

$$\underbrace{d^{(k)T} (g^{(k+1)} - g^{(k)})}_{\text{HS Denominator}} \xrightarrow{\text{Exact Line Search}} \underbrace{\|g^{(k)}\|^2}_{\text{PR Denominator}}$$

The Takeaway

Whenever an **exact line search** is performed, the Hestenes-Stiefel and Polak-Ribière methods are mathematically equivalent ($\beta_k^{HS} \equiv \beta_k^{PR}$). They will always produce the exact same path!

Algorithmic Characteristics & Final Comments

1. Fletcher–Reeves (FR) Divergence:

- ▶ Because FR relies **only on gradient norms** and ignores the directional change ($g^{(k+1)} - g^{(k)}$), β_1^{FR} becomes inappropriately large (≈ 1.0665).
- ▶ This pushes FR off the efficient path. While mathematically proven to converge, it is highly prone to **jamming** (taking many tiny, inefficient steps) in non-quadratic problems.

2. PR and Automatic Restart:

- ▶ Notice that $\beta_2^{PR} \approx 0.0129$ is very small.
- ▶ PR has a built-in safety mechanism: when a step makes little progress, $g^{(k+1)} \approx g^{(k)}$, causing the numerator to drop and $\beta_k^{PR} \rightarrow 0$.
- ▶ This effectively "restarts" the algorithm by turning $d^{(k+1)}$ back into the pure steepest descent direction ($-g^{(k+1)}$), elegantly preventing the jamming issues seen in FR.

Conclusion: Which Method Should We Prefer?

Based on our computations and theoretical properties, we can make the following practical observations:

Hestenes–Stiefel (HS):

- ▶ Uses both gradient and direction information.
- ▶ Highly **adaptive** to the local geometry of the problem.
- ▶ Generally very effective and robust in practice.

Polak–Ribière (PR):

- ▶ Often behaves like HS
- ▶ Features an implicit **automatic restart mechanism** (avoids jamming).
- ▶ Widely used in applications

Fletcher–Reeves (FR):

- ▶ Simpler formula (relies only on gradient norms).
- ▶ Possesses strong **global convergence** proofs.
- ▶ Can be less adaptive in practice, sometimes leading to inefficient steps.

Revisiting Example 5: What Happens on a Quadratic Function?

Question: What happens if we apply the nonlinear CG formulas

$$\beta_k^{FR}, \quad \beta_k^{PR}, \quad \beta_k^{HS}$$

to the strictly quadratic function from Example 5?

Orthogonality of Successive Gradients

For a strictly quadratic function

$$f(x) = \frac{1}{2}x^T Qx - b^T x$$

with exact line search, successive gradients are orthogonal:

$$g^{(k+1)T} g^{(k)} = 0.$$

This single property greatly simplifies the nonlinear CG formulas.

Revisiting Example 5: Why the Formulas Coincide

Using

$$g^{(k+1)T} g^{(k)} = 0,$$

let us simplify the Polak–Ribière formula:

$$\begin{aligned}\beta_k^{PR} &= \frac{g^{(k+1)T} (g^{(k+1)} - g^{(k)})}{g^{(k)T} g^{(k)}} \\ &= \frac{g^{(k+1)T} g^{(k+1)} - \cancel{g^{(k+1)T} g^{(k)}}}{g^{(k)T} g^{(k)}} = \frac{g^{(k+1)T} g^{(k+1)}}{g^{(k)T} g^{(k)}} = \beta_k^{FR}.\end{aligned}$$

Similarly, under exact line search for quadratic problems, the HS formula also reduces to the same value.

For strictly quadratic problems with exact line search,

$$\beta_k^{HS} = \beta_k^{PR} = \beta_k^{FR}.$$

So nonlinear CG reduces to the classical conjugate gradient method.

Nonlinear CG on Quadratic Functions

The Convergence of Formulas:

When we apply the nonlinear formulas $(\beta_k^{HS}, \beta_k^{PR}, \beta_k^{FR})$ to a strictly quadratic problem with exact line search, they all become **identical**.

The "Quadratic Harmony" Properties

In the quadratic case with exact line search, two critical properties emerge:

1. **Orthogonal Gradients:** $g^{(k+1)T} g^{(k)} = 0$
2. **Direction Orthogonality:** $g^{(k+1)T} d^{(k)} = 0$

Under these conditions, the complex nonlinear logic collapses back into the classical Conjugate Gradient method.

Why the Formulas Coincide

1. From PR to FR:

$$\beta_k^{PR} = \frac{g^{(k+1)T}(g^{(k+1)} - g^{(k)})}{g^{(k)T}g^{(k)}} = \frac{\|g^{(k+1)}\|^2 - \cancel{g^{(k+1)T}g^{(k)}}^0}{\|g^{(k)}\|^2} = \beta_k^{FR}$$

2. From HS to PR: Recall the HS denominator: $d^{(k)T}(g^{(k+1)} - g^{(k)})$.
Using $g^{(k+1)T}d^{(k)} = 0$ and substituting $d^{(k)} = -g^{(k)} + \beta_{k-1}d^{(k-1)}$:

$$\text{Denom} = \cancel{d^{(k)T}g^{(k+1)}}^0 - d^{(k)T}g^{(k)} = -(-g^{(k)} + \dots)^T g^{(k)} = \|g^{(k)}\|^2$$

Conclusion

For strictly quadratic problems with exact line search:

$$\beta_k^{HS} \equiv \beta_k^{PR} \equiv \beta_k^{FR} = \frac{\|g^{(k+1)}\|^2}{\|g^{(k)}\|^2}$$

The three methods produce the **exact same search directions**.

Appendix: Gram–Schmidt Orthogonalization

Given linearly independent vectors

$$v_1, v_2, \dots, v_m \in \mathbb{R}^n,$$

the Gram–Schmidt process constructs an orthogonal set

$$u_1, u_2, \dots, u_m.$$

Step 1:

$$u_1 = v_1.$$

Step 2:

$$u_2 = v_2 - \frac{u_1^T v_2}{u_1^T u_1} u_1.$$

General step: for $k \geq 2$,

$$u_k = v_k - \sum_{i=1}^{k-1} \frac{u_i^T v_k}{u_i^T u_i} u_i.$$

Then $u_i^T u_j = 0$, $i \neq j$.

If needed, an orthonormal set is obtained by normalizing:

$$e_k = \frac{u_k}{\|u_k\|}.$$

Appendix: Gram–Schmidt Example

Consider the vectors

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

We apply the Gram–Schmidt process.

First vector:

$$u_1 = v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Second vector:

$$u_2 = v_2 - \frac{u_1^T v_2}{u_1^T u_1} u_1.$$

Compute

$$u_1^T v_2 = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1, \quad u_1^T u_1 = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 2.$$

Hence

$$u_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \end{bmatrix}.$$

Appendix: From Gram–Schmidt to Q-Conjugacy

In the standard Gram–Schmidt process, orthogonality is defined by the usual inner product

$$\langle x, y \rangle = x^T y.$$

For conjugate direction methods, we replace this with the Q -inner product

$$\langle x, y \rangle_Q = x^T Q y.$$

Thus, the construction of Q -conjugate directions is analogous to Gram–Schmidt orthogonalization, but with respect to $\langle x, y \rangle_Q$.

The corresponding formula is

$$d^{(k)} = v^{(k)} - \sum_{i=0}^{k-1} \frac{(d^{(i)})^T Q v^{(k)}}{(d^{(i)})^T Q d^{(i)}} d^{(i)}.$$

Appendix: Understanding "Jamming"

The Achilles' Heel of FR: What is Jamming?

It is a phenomenon where an optimization algorithm gets trapped in a vicious cycle, taking an endless sequence of microscopic steps without making meaningful progress toward the minimum.

Why does it happen in FR?

- ▶ If the algorithm takes a poor step (due to an inexact line search), the new search direction $d^{(k+1)}$ can become almost perpendicular to the true steepest descent direction $-g^{(k+1)}$.
- ▶ Because the FR formula relies **only** on the ratio of gradient norms ($\|g^{(k+1)}\|^2 / \|g^{(k)}\|^2$), it cannot mathematically detect this geometric "trap".
- ▶ PR and HS avoid this because they look at the *difference* between gradients, naturally resetting when progress halts. FR just keeps building on the flawed past directions.

The Practical Fix: To prevent jamming, software implementations of FR usually force a **manual restart** (setting $\beta_k = 0$) every n iterations to clear the algorithm's memory.

Appendix: The Strong Wolfe Conditions

Ensuring "Good Enough" Steps in Line Search

In nonlinear optimization, we rarely perform an *exact* line search. Instead, we accept a step length α_k if it satisfies the **Strong Wolfe conditions**, which consist of two rules ($0 < c_1 < c_2 < 1$):

1. Sufficient Decrease (Armijo Condition):

$$f(x^{(k)} + \alpha_k d^{(k)}) \leq f(x^{(k)}) + c_1 \alpha_k g^{(k)T} d^{(k)}$$

The step must actually lower the function value by a proportional amount. Just going slightly "down" isn't enough; it must go down sufficiently relative to the initial slope.

2. Strong Curvature Condition:

$$|g^{(k+1)T} d^{(k)}| \leq c_2 |g^{(k)T} d^{(k)}|$$

The slope at the new point must be significantly flatter (closer to zero) than at the starting point. The absolute value ensures we don't step too far past the local minimum into a steep uphill area.

Note: These conditions act as a mathematical safety net. They are exactly what provides the FR algorithm its rigorous global convergence guarantee.